

CODE VALIDATION THROUGH MACHINE LEARNING. A LEFT SHIFT FOCUS STRATEGY

This Thesis is submitted to the Department of Computer Science as partial fulfillment of Master of Science in Computer Science/ Data Science degree

by

Syed Jehanzeb

Supervised by
Dr. Faisal Iradat
Assistant Professor
Department of Computer Science
School of Mathematics and Computer Science (SMCS)
Institute of Business Administration (IBA), Karachi

Fall 2024
Institute of Business Administration (IBA), Karachi, Pakistan

CODE VALIDATION THROUGH MACHINE LEARNING. A LEFT SHIFT FOCUS STRATEGY

This Thesis is submitted to the Department of Computer Science as partial fulfillment of Master of Science in Computer Science/ Data Science degree

by

Syed Jehanzeb
(Student Number: 04879)

Committee:

Dr. Faisal Iradat
Supervisor, IBA Karachi

Dr. Tariq Mahmood
Examiner – I, IBA Karachi

Dr. First Name Last Name
Examiner – II, University Affiliation

Dr. Shakeel Ahmed Khoja
Dean, SMCS

Copyright: **2025, Syed Jehanzeb**
All Rights Reserved

Contents

1	Introduction	1
1.1	Overview	1
1.2	Motivation	3
1.3	Problem Statement	3
1.4	Research Gap and Proposed Direction	4
1.5	Research Objectives	5
1.6	Research Questions	5
1.7	Scope and Limitations	5
1.8	Significance of the Study	6
1.9	Thesis Organization	7
1.10	Mind Map of the Thesis	7
2	Background	9
2.1	Foundations of API Vulnerability Research	9
2.2	Machine Learning for Vulnerability Detection	10
2.2.1	Deep Learning in Vulnerability Detection	12
2.3	Explainable AI in Security Applications	13
2.4	Comparative Analysis of Related Work	13
3	Methodology	15
3.1	Model Selection and Architecture	15
3.1.1	Machine Learning Model Selection and Architecture	20
4	Training Process	23
4.1	Data Splitting	23
4.2	Feature Representation	23
4.3	Model Training and Hyperparameter Tuning	24
4.4	Evaluation Metrics	25

5	Results	27
5.1	Logistic Regression	27
5.2	Random Forest Classifier	27
5.3	Support Vector Machine (SVM)	28
5.4	Gradient Boosting Classifier	28
5.5	Extra Trees Classifier	28
5.6	XGBoost Classifier	28
5.7	FLAML AutoML	29
5.8	Summary of Findings	29
6	Validation through Explainable artificial intelligence (XAI)	30
6.1	Motivation for XAI	30
6.2	SHAP for Feature Importance Analysis for Vulnerability Diagnosing	31
6.3	Impact of XAI on Model Optimization for Vulnerability Diagnosing	33
6.4	Discussion and Conclusion	33
7	Discussion	34
7.1	Model Performance Evaluation	34
7.2	Impact of Explainable AI	35
8	Limitations and Future Work	36
9	Conclusion	37

List of Abbreviations

API	Application Programming Interfaces
ASTs	Abstract Syntax Trees
CFGs	Control Flow Graphs
CNN	Convolutional Neural Networks
FLAML	A Fast Library for Automated Machine Learning & Tuning
GNNs	Graph Neural Networks
LLMs	Large Language Models
LSTM	Long Short-Term Memory
ML	Machine Learning
OWASP	Open Web Application Security Project
PDGs	Program Dependency Graphs
RBF	Radial Basis Function
RNNs	Recurrent Neural Networks
SDLC	Software Development Life Cycle
SHAP	SHapley Additive exPlanations
SVM	Support Vector Machines
TF-IDF	Term Frequency-Inverse Document Frequency
XAI	Explainable Artificial Intelligence

Abstract

The digital transformation relies heavily on Application Programming Interfaces for information exchange. However, the projected surge in API use, often unmanaged introduces substantial security risks. The OWASP API Security Top Ten list highlights the critical vulnerabilities surrounding unmanaged APIs, guiding research and industry efforts. The direction of current research is more focused on shift right security paradigm leaving room for shift left security i.e. security early in software development life cycle. Catching security issues early in SDLC requires data on best practices, which then can be learned through machine learning assuming availability of right data. Existing research is struggling to identify the right data.

The research uses OWASP API Top Ten Vulnerabilities to identify the exploits and develop focused security related codes. A custom dataset of 60 labeled python snippets was created focusing on API Vulnerabilities, which were then used to learn through Machine Learning algorithms. Various ML models were evaluated, with ensemble methods outperforming single classifiers. The Extra Trees Classifier achieved the highest test accuracy (83.3

To identify the factors influencing the machine learning results, we used eXplainable Artificial Intelligence tools such as SHapley Additive exPlanations or SHAP which was employed to provide transparent feature importance analysis, which among other things identified that in most cases security is just assumed to be managed and not focused upon.

Keywords: *Explainable Artificial Intelligence, API Vulnerability Detection, Machine Learning, SHapley Additive exPlanations, Feature Importance Analysis, Model Interpretability, Security, Shift-Left Security*

Chapter 1

Introduction

This chapter lays the groundwork for our investigation into API security, starting with an overview of the digital landscape and the pivotal role of APIs. Following this, we delve into the motivation behind this research, highlighting the escalating concerns surrounding API vulnerabilities. We then clearly articulate the problem statement, identifying the specific challenge this thesis aims to address, before outlining the existing research gap and the proposed direction of our work. The chapter concludes by defining the research objectives, posing the guiding research questions, establishing the scope and limitations of our study, and finally, underscoring the significance of this research.

1.1 Overview

The current digital transformation sweeping the globe is unprecedented. Content generation, with an ever-increasing pace, often exceeds the rate of human consumption. The affordances of content consumption offer insights and knowledge previously unimaginable, fueling the further generation of content in a self-sustaining cycle.

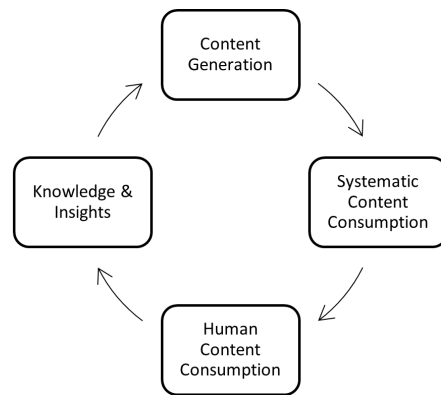


Figure 1.1: Content Generation Cycle

Systematic Content consumption is the driving force behind Content Generation, where independent applications systematically exchange information without the need for human interface. It leads to generating new content and insights which fuels further content generation 1.1 . Application Programming Interfaces (APIs) serve as the medium of systematic communication for independent applications, enabling seamless data exchange without intricate understanding of each other's internal workings.

The benefits of APIs are evident in their contribution to the efficient data exchange, which not only supports the growth of information but also streamlines organizational operations. However, it is crucial to acknowledge that these advantages coexist with concerns regarding the potential compromise of the vast movement of data. Security breaches, often linked to vulnerable APIs, can expose critical business information, and disrupt the normal functioning of an organization.

As organizations continue to modernize their app portfolios, the number of APIs in use is projected to exceed one billion by 2031 ([6]). Although API allows for a certain level of control, unmanaged APIs introduce significant security risks, including but not limited to authentication, authorization, input validation, data exposure and other security misconfigurations, that cannot be ignored. According to Gartner, by 2025, 50% of all APIs are expected to be unmanaged Gartner [8].

These unmanaged APIs can be exploited by attackers to gain unauthorized access, leading to data breaches, service disruptions, and potential compliance violations. Organizations must address these risks by implementing robust API management and security strategies to safeguard their systems.

1.2 Motivation

Realizing threats to APIs, the Open Web Application Security Project (OWASP), a recognized authority in software security, launched the API Security Top Ten list [7] in 2019. OWASP which was founded in 2001 by Mark Curphey, a software security expert, as a response to the growing need for standardized security practices in web application development. At the time, web security was often an afterthought and there was no widely accepted framework for securing web applications. OWASP was established as a non-profit, open community where developers, security researchers, and organizations could collaborate to improve web security. In 2003, OWASP released its first OWASP Top 10 list, a document that highlighted the most critical security risks for web applications. Between 2004 and 2010, OWASP expanded rapidly. The project evolved from a small initiative into an internationally recognized authority on web application security. By releasing regular updates to the OWASP Top Ten along with important tools, frameworks, and standards, it became the industry standard setter. This list quickly became the industry standard for web security assessments and best practices. In recent times, OWASP is regarded as a global leader in web application security. It is widely referenced in security policies, compliance regulations, and industry best practices. The OWASP community, consisting of hundreds of security professionals, continues to drive innovative solutions for modern cybersecurity challenges ([1]). With the rise of AI, machine learning, and quantum computing, OWASP is expected to play a crucial role in shaping the future of web and API security by continuously evolving its methodologies and frameworks.

The OWASP API Security Top Ten ([7]) the industry-recognized guide for the most critical API vulnerabilities list Broken Authentication and Broken Object Level Authorization as the top two critical vulnerabilities on the list. The list contains an in-depth analysis, explanation, scenarios and preventative measure for each of the top 10 vulnerabilities. This standardization of vulnerability data has laid down a blueprint for other developers and organizations working to safeguard their data.

1.3 Problem Statement

This research aims to analyze the efficacy of Machine Learning Algorithms for a proposed left-shift security solution which is analyzing and catching vulnerabilities in source code, that is to say programming code before compilation. The

source of vulnerabilities will be OWASPs API Top Ten list.

1.4 Research Gap and Proposed Direction

In part I of this thesis, a review was conducted to evaluate the current direction of research in API security. The findings highlighted a predominant focus on shift-right security. Simply, the security arena is focused more on testing the product rather than building security within the production cycle. This thesis, however, is focused on the premise that there is comparatively less emphasis on shift-left security in the cybersecurity domain, resulting in a significant research gap.

Shift-left security ([9]) involves integrating security measures early in the software development lifecycle, focusing on identifying and addressing vulnerabilities during the design, development, and testing stages. This proactive approach ensures that security issues, such as broken authentication, insecure data exposure, and improper input validation, are mitigated before products are deployed. By incorporating techniques such as static code analysis ([27]), security testing in CI/CD pipelines, and secure API design principles, shift-left security reduces the attack surface, lowers costs associated with post-production fixes, and ensures that the code is inherently more secure.

Shift-right security ([23]) focuses on monitoring and responding to security threats after APIs are deployed in production ([24]). It emphasizes real-time detection, logging, and incident response, ensuring that vulnerabilities or attacks, such as zero-day exploits or runtime misconfigurations, are identified and addressed quickly. Unlike shift-left, which prevents issues during development, shift-right assumes that security incidents will occur and focuses on minimizing damage through continuous monitoring ([23]), user behavior analysis, and rapid recovery mechanisms. This approach complements shift-left by providing ongoing protection and adaptability in dynamic, real-world environments.

This thesis will explore the solution associated with Shift Left Security. The concept of Shift Left Security is to integrate security measures early in the development life-cycle of software. The focus of this paper is to enhance Shift Left Security during API development by leveraging **Machine Learning (ML)** models to detect vulnerabilities in API authentication code. The OWASP API Top Ten ([7]) vulnerabilities will serve as a basis for Machine Learning algorithms. To complement this, we also incorporate **Explainable AI (XAI)** techniques adding transparency and interpretability into the decision-making process of the models. XAI techniques are essential for making AI-driven security solutions more

accessible and trustworthy by enabling developers to understand and explain the reasoning behind vulnerability predictions made by the models.

Ultimately, this work seeks to explore the efficacy of Machine Learning models for AI-powered security solutions, complying with OWASP guidelines, which not only improve vulnerability detection but also ensure that the underlying models are interpretable and actionable by developers, promoting the widespread adoption of AI-driven Shift Left Security practices.

1.5 Research Objectives

This research aims to enhance shift-left security by leveraging machine learning and explainable AI to detect and interpret vulnerabilities in API code. The specific objectives are:

- To develop a machine learning model that assesses the security level of API authentication code, addressing the detection of common vulnerabilities.
- To integrate Explainable AI (XAI) techniques to identify the key features influencing the model's output.
- To improve the interpretability and trust of AI-based security tools for developers by visualizing the rationale behind predictions.

1.6 Research Questions

To achieve the research objectives, this study is guided by the following key questions:

RQ1: How does Machine Learning help identify the code security level?

RQ2: Can Explainable AI (XAI) identify the key points influencing the output of Machine Learning Models?

1.7 Scope and Limitations

Scope

This study focuses on authentication in Python, which has consistently ranked among the top OWASP API vulnerabilities. The dataset used consists of 60

custom-labeled Python code snippets simulating API authentication flaws, with each entry tagged by primary and secondary vulnerability categories. It includes both insecure and corrected examples, ensuring balance. Inspired by the OWASP API Top Ten, the dataset supports focused analysis on authentication-related weaknesses. While limited to authentication, the dataset lays groundwork for future Shift Left Source Code solutions and broader application to other OWASP categories like authorization and data validation.

Limitations

While the study demonstrates promising results, several limitations should be addressed in future work:

- **Dataset Limitation:** The dataset used in this study contained only 60 labeled code snippets. Although it was sufficient for initial exploration, a larger and more diverse dataset would be necessary to validate the results and improve model robustness.
- **Model Variability:** While ensemble models like Extra Trees performed well, other models like XGBoost and SVM showed inconsistent performance. Further tuning and experimentation with different hyperparameter configurations might help enhance their stability and predictive power.
- **Scalability of XAI Techniques:** As the dataset grows, the computational burden of XAI methods like SHAP may increase. Exploring other model-agnostic interpretability techniques, such as LIME, or improving the scalability of SHAP would be valuable.
- **Hybrid Models:** Future work could explore combining the strengths of both unsupervised and supervised learning to create hybrid models that can effectively detect vulnerabilities while maintaining interpretability.

1.8 Significance of the Study

This study highlights the importance of interpretability in security-related AI applications, providing developers with actionable insights into the features driving vulnerability predictions. By combining high-performing models with transparent, explainable techniques, we can not only improve model accuracy but also

ensure that security decisions are well-understood, trusted, and actionable. The integration of SHAP-based Explainable AI (XAI) techniques into the model evaluation process has proven invaluable. Transparent visualizations of feature contributions facilitate better decision-making and model refinement, supporting the development of robust API vulnerability detection frameworks. These advancements enhance AI-driven security solutions by fostering trust, providing transparency, and ultimately guiding developers in building more secure systems.

1.9 Thesis Organization

- Chapter 2: Background
- Chapter 3: Methodology
- Chapter 4: Training & Dataset
- Chapter 5: Results
- Chapter 6: Validation through Explainable artificial intelligence (XAI)
- Chapter 7: Discussion
- Chapter 8: Limitations & Future Work
- Chapter 9: Conclusions

1.10 Mind Map of the Thesis

The figure 1.2 explains the development of the thesis. The research base was started from OWASP API Top Ten list [7] with review of current direction on API research, resulting in potential Machine Learning models to work on the dataset. This went in parallel to understanding and identifying the exploits referred through OWASP API Top Ten. The exploits were used to develop codes secure and insecure codes. The codes were tested in the machine learning model. The result were passed through Explainable AI models for interpretation.

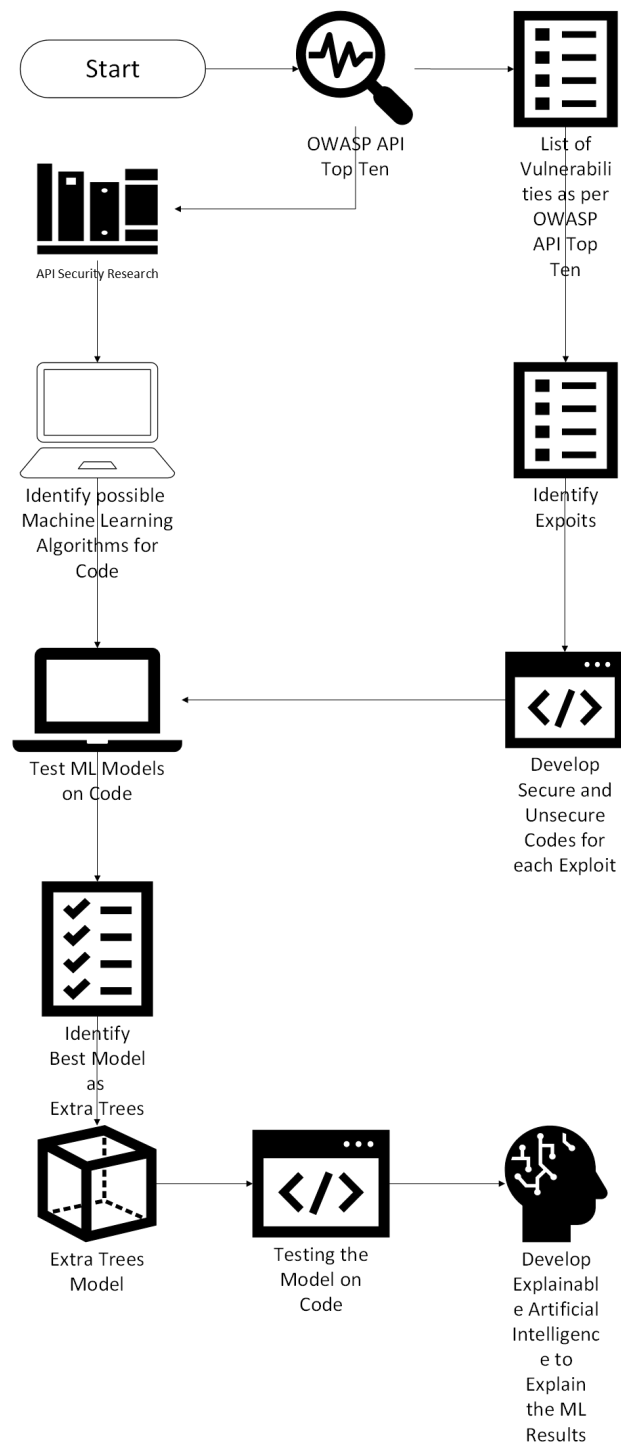


Figure 1.2: Mind Map of the Thesis

Chapter 2

Background

2.1 Foundations of API Vulnerability Research

Building security or quality within the production cycle is not new. This has been in practice since the 1980s in Japan and was standardized by W. Edwards Deming in the famous 1987 paper. He emphasized in his quality management philosophy that quality should be built into the product itself [16]. Security is an important part of the quality of a product [3]. The age-old practice of testing the quality of a product after it has been developed is costly and inefficient. If security is built into the product in the first place, it saves a lot of testing and revision time, eventually speeding up the product development lifecycle.

Traditional methods for detecting software vulnerability have mainly relied on static and dynamic analysis tools. Static analysis tools, such as *Flawfinder*, *RATS*, and *CPPCHECK*, have been effective in identifying vulnerabilities in programming languages such as C / C++ and Java. However, these tools often suffer from high false positive rates [26, 4]. Dynamic analysis tools, such as *Sydr-Fuzz*, are useful in post-deployment phases but are less efficient for early-stage detection [24]. A brief comparison of Static and Dynamic Testing is reproduced below 2.1.

Despite their utility, these traditional tools lack the ability to generalize across diverse code-bases and often require significant manual effort to address detected issues. These limitations have paved the way for more advanced methods based on machine learning and deep learning [25].

Table 2.1: Comparison of Static and Dynamic Analysis Tools

Aspect	Static Analysis Tools	Dynamic Analysis Tools
Key Points	Analyzes source code without executing it	Analyzes program behavior during execution
Benefits	Detects potential issues early No need to execute the program Comprehensive coverage of code paths	Identifies runtime issues like memory leaks Real-world input testing Better insights into program behavior
Challenges	May generate false positives Limited to code-level insights Cannot detect runtime errors	Requires execution environment setup Performance overhead during analysis May miss dormant code paths

2.2 Machine Learning for Vulnerability Detection

Machine learning has emerged as a promising alternative to traditional methods. Machine Learning utilizes data-driven techniques to identify patterns and anomalies in code that may indicate vulnerabilities.

Several Machine Learning techniques have been applied to this field, with varying degrees of success. Supervised learning approaches are often used, where labeled datasets of vulnerable and non-vulnerable code are employed to train classifiers. Techniques like logistic regression, support vector machines (SVM), and decision trees are among the traditional approaches. A summary of potential machine learning algorithms are described in 2.2

Marjanov et al. [13] discuss ML pipelines for vulnerability detection, emphasizing the importance of high-quality datasets and the challenges of overfitting and generalization. Token-based sequence processing and statistical feature extraction are commonly used in ML-based approaches.

Integrating AI into vulnerability detection has shown promising results. Rajapaksha et al. [18] proposed a machine learning-based method for detecting vulnerabilities in C/C++ applications. Their approach achieved an F1-Score of 0.96 in binary classification and 0.85 in multi-class classification based on CWE-IDs. Additionally, they applied Explainable AI techniques to assist developers in identifying and understanding vulnerable code segments. The models were trained on datasets containing actual source code, ensuring high generalizability.

Table 2.2: Summary of Machine Learning Algorithms with Brief Descriptions

Algorithm	Brief Description	Strengths	Weaknesses	Best Use Cases
Logistic Regression	Predicts probabilities for binary classification using a sigmoid function.	Simple, interpretable, provides probabilities.	Assumes linear relationships, struggles with complex patterns.	Binary classification tasks, medical diagnoses.
Support Vector Machines (SVM)	Finds an optimal hyperplane to separate classes in feature space.	Effective for small, high-dimensional datasets.	Computationally expensive, sensitive to kernel selection.	Text/image classification, small datasets.
Decision Trees	Creates a tree by iteratively splitting data based on feature values.	Simple, interpretable, handles mixed data types.	Overfitting, unstable (small data changes affect results).	Quick prototyping, interpretable decision-making.
XGBoost	A boosting algorithm that builds sequential trees to optimize errors.	High accuracy, efficient, robust.	Complex to tune, prone to overfitting without care.	Competitive machine learning tasks, structured data.
Extra Trees	Randomized decision tree splits increase generalization.	Fast training, reduces overfitting.	Random splits may yield less optimal models.	Large datasets, feature selection.

2.2.1 Deep Learning in Vulnerability Detection

Deep Learning (DL) is a subset of machine learning that uses neural networks with multiple layers to automatically learn representations from data. Deep Learning models can extract high-level, abstract features from raw input data, unlike traditional machine learning models, that often require extensive feature engineering. This capability makes Deep Learning particularly suitable for modeling complex patterns in data, such as code structures and contextual relationships in software. Deep learning models have gained traction due to their ability to model complex patterns.

- **Sequence-Based Models:** Sequence-based models treat source code as a series of tokens or instructions and analyze temporal or contextual relationships between them. By leveraging architectures such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and transformers, these models excel at understanding the sequential nature of code execution and dependencies. Models such as *VulDeePecker* and *SySeVR* leverage RNNs and transformers to process code sequences, demonstrating strong performance in contextual vulnerability detection [11, 12].
- **Graph-Based Models:** Graph-based models represent source code as graphs, such as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), or Program Dependency Graphs (PDGs), capture the structural and syntactic relationships in the code. Graph Neural Networks (GNNs) are then used to process these graph representations and extract meaningful embeddings. For instance, *GraphcodeBERT* combines graph-based embeddings with pre-trained transformer models to capture both semantic and structural aspects of code, while *Vul-LMGNN* [12] integrates graph embeddings with language models to achieve excellent performance in vulnerability detection. These models are useful for identifying complex vulnerabilities related to data flows, control flows, or multi-function interactions within large codebases.

Russell et al. [21] developed a large-scale function-level vulnerability detection system using convolutional and recurrent neural networks. By standardizing source code representations with a custom lexer, the authors minimized noise and improved feature extraction. The CNN+RF hybrid model achieved superior performance compared to traditional static analysis tools like Clang and Flawfinder. This study highlights the potential of combining deep learning with ensemble

methods for precise and scalable vulnerability detection. Deep learning has indicated promise in enhancing vulnerability detection through feature representation learning. Russell et al. [21] demonstrated that combining CNNs and random forests improves the granularity and scalability of vulnerability detection systems.

2.3 Explainable AI in Security Applications

There is an importance of integrating Explainable AI techniques for improving transparency and interpretability in AI-driven vulnerability detection systems. This remains a critical area for future research, alongside the development of more robust hybrid methods and the creation of comprehensive benchmark datasets to evaluate their performance.

In later stages of this thesis, SHAP (SHapley Additive exPlanations) is employed to visualize the influence of specific code tokens on model predictions. This interpretability bridges the gap between high-performing models and real-world developer trust, aligning with the need identified in existing literature.

2.4 Comparative Analysis of Related Work

Hybrid methods that combine token-based and graph-based techniques are gaining attention. For example, *Vul-LMGNN* combines graph neural networks with pre-trained code language models, effectively utilizing both structural and contextual features [12]. Similarly, *EnStack* employs ensemble techniques with pre-trained language models like *CodeBERT* to enhance vulnerability detection [22].

Recent studies have highlighted the potential of large language models (LLMs), such as *GPT-2*, *RoBERTa*, and *CodeT5*, in vulnerability detection. These models have been shown to outperform traditional graph neural networks when provided with sufficiently large datasets, emphasizing the need for domain-specific pre-training [5, 12].

Table 2.3: Summary of Hybrid Methods Combining Token-Based and Graph-Based Techniques

Method	Technique	Highlights
Vul-LMGNN	Combines graph neural networks with pre-trained code language models	Utilizes both structural and contextual features for enhanced vulnerability detection.
EnStack	Employs ensemble techniques with pre-trained language models like CodeBERT	Enhances vulnerability detection by leveraging pre-trained language model capabilities.

Chapter 3

Methodology

The thesis is divided into two parts, Systematic Literature Review to identify the gap and to conduct Experimental Research to test the solution to plug the gap. The next section will explain the process carried to conduct the Experimental Research.

3.1 Model Selection and Architecture

The aim of the second part of the thesis is to explore the ability Machine Learning models in detecting vulnerabilities in source code. We will do that within the context of OWASP's Top Ten API Security vulnerabilities. This study employs a structured approach to develop a machine learning pipeline for the classification of coding language, focusing on dataset creation, machine learning model selection, its training process, and explainable artificial intelligence (XAI). The methodology is divided into four phases:

1. Dataset creation and transformation
2. Machine Selection and Architecture
3. Model Training Process
4. Explainable artificial intelligence (XAI)

Dataset

The dataset was created with the help of a security professional specifically for this thesis. The area targeted for the thesis is Authentication in Python, which has remained the top three in the past years [?]. The dataset contains a variety of Python code simulating different vulnerabilities in API authentication code, focusing on authentication-based features such as logging in and signing up. The types and variety of the dataset was inspired from OWASP API Top Ten vulnerability list [7]. The dataset consists of 60 code snippets, each labeled with primary and secondary vulnerability categories. It is compact yet diverse, allowing for a focused analysis on authentication vulnerabilities.

$$\begin{array}{ccccccc} 10 & \times & 3 & \times & 2 & = & 60 \\ \textit{(OWASP API} & & \textit{(Vulnerabilities} & & \textit{(Secure/Insecure} & & \textit{(Code} \\ \textit{Top Ten)} & & \textit{Types)} & & \textit{Versions)} & & \textit{Snippets)} \end{array}$$

Each entry has a primary label linking it to an OWASP API Security Vulnerability, along with a secondary label highlighting the specificity of the vulnerability, such as `Inadequate user validation`. The dataset contains two entries per secondary label: one with Vulnerability Code 3.1 and the other as Secure Code 3.2. This results in a balanced dataset, with each sub-label containing evenly distributed code for secure and insecure practices.

While focused on authentication, this dataset may lay groundwork for future Shift Left Source Code solutions, due to its custom nature and the fact that we were unable to find many research papers that exhaustively used Machine Learning techniques for vulnerability detection in code having vulnerabilities with respect to OWASP's Top Ten API Security Guidelines, which could be expanded to include other vulnerabilities like authorization and data validation, broadening its applicability.

```
1 from flask import Flask, request, jsonify
2 app = Flask(__name__)
3 # Mock user data
4 users = {
5     1: {"name": "Alice", "email": "alice@example.com"},
6     2: {"name": "Bob", "email": "bob@example.com"}
7 }
8 @app.route('/user/<int:user_id>', methods=['GET'])
```

```

9 def get_user(user_id):
10     user = users.get(user_id)
11     if user:
12         return jsonify(user)
13     else:
14         return jsonify({"error": "User_not_found"}), 404
15     if __name__ == '__main__':
16         app.run(debug=True)

```

Listing 3.1: Vulnerable Code

```

1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 # Mock user data
6 users = {
7     1: {"name": "Alice", "email": "alice@example.com"},
8     2: {"name": "Bob", "email": "bob@example.com"}
9 }
10
11 # Mock API keys for users
12 api_keys = {
13     "key_for_alice": 1,
14     "key_for_bob": 2
15 }
16
17 def get_user_id_from_api_key(api_key):
18     return api_keys.get(api_key)
19
20 @app.route('/user/<int:user_id>', methods=['GET'])
21 def get_user(user_id):
22     api_key = request.headers.get('X-API-Key')
23     if not api_key:
24         return jsonify({"error": "Missing API key"}), 401
25
26     requesting_user_id = get_user_id_from_api_key(api_key)
27     if not requesting_user_id:
28         return jsonify({"error": "Invalid API key"}), 403
29
30     if requesting_user_id != user_id:
31         return jsonify({"error": "Unauthorized access"}), 403
32
33     user = users.get(user_id)
34     if user:

```

```

35         return jsonify(user)
36     else:
37         return jsonify({"error": "User not found"}), 404
38
39 if __name__ == '__main__':
40     app.run(debug=True)

```

Listing 3.2: Secure Code

In 3.1, an API endpoint allows users to access details of any user by specifying the user ID in the URL. The endpoint fails to check whether the requesting user is authorized to access the requested user's details. Code length is also observed categorized by primary vulnerabilities 3.1

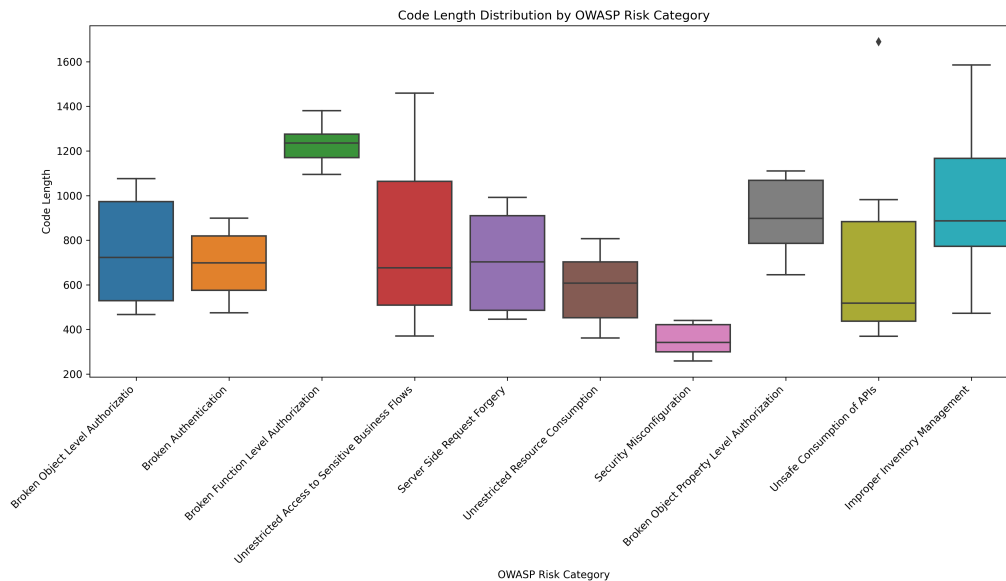


Figure 3.1: Code Length Distribution by OWASP Risk Category

Code comments were removed as they could indirectly act as indicators for vulnerabilities, thereby biasing the theoretical AI solution for source code discussed later in this paper. Additionally, stopwords were filtered out to reduce complexity in the models.

Vectorization and Label Encoding The dataset underwent vectorization and label encoding. Vectorization is a preprocessing step that transforms textual data

into numerical representations suitable for machine learning model input. Two vectorization techniques were employed:

1. **Count Vectorization:** This method captures the frequency of each token within a code snippet, resulting in a sparse matrix representation. It provides a direct representation of token occurrence without considering their importance across the dataset.
2. **TF-IDF (Term Frequency-Inverse Document Frequency):** This method assesses the relative importance of tokens by balancing their frequency in individual snippets with their prevalence across the entire dataset. TF-IDF assigns higher weights to tokens that are unique to specific code snippets while down-weighting commonly occurring ones, such as programming keywords.

Both vectorization methods 3.3 utilized the filtered tokens generated during preprocessing. The transformation steps were as follows:

```
1 # Count Vectorization
2 vectorizer = CountVectorizer()
3 count_vectors = vectorizer.fit_transform(
4     df['filtered_tokens'].apply(lambda tokens: ' '.join(
5         tokens))
6 )
7 # TF-IDF Vectorization
8 tfidf_vectorizer = TfidfVectorizer()
9 tfidf_vectors = tfidf_vectorizer.fit_transform(
10     df['filtered_tokens'].apply(lambda tokens: ' '.join(
11         tokens))
12 )
```

Listing 3.3: Vectorization Process

The dataset's `Result` column, which indicates whether a code snippet contains a vulnerability or its resolved counterpart, was numerically encoded for compatibility with machine learning algorithms. The label encoding 3.4 ensured that models could process the categorical data. A binary mapping was applied: `Error` was encoded as 0, and `Resolved` was encoded as 1. The following Python snippet illustrates the process:

```

1 from sklearn.preprocessing import LabelEncoder
2
3 # Encode the 'Result' column
4 le = LabelEncoder()
5 df['Result_encode'] = le.fit_transform(df['Result'])

```

Listing 3.4: Label Encoding

The vectorization and label encoding steps were essential in preparing the dataset for downstream tasks, ensuring model compatibility and efficient processing for machine learning models.

Stopwords Used for Filtering A custom stopwords list 3.5 was used to filter out irrelevant tokens, such as common programming keywords and special characters. This helped reduce noise in the dataset, improving the performance of machine learning models. The stopwords used for filtering are as follows:

```

1 stopwords = [
2     '\n', 'utf-8', 'False', 'None', 'True', 'and', 'as', '
3     assert', 'async', 'await', 'break', 'class', '
4     continue',
5     'def', 'del', 'elif', 'else', 'except', 'finally', 'for'
6     ', 'from', 'global', 'if', 'import',
7     'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
8     'raise', 'return', 'try', 'while', 'with',
9     'yield', '+', '-', '*', '/', '%', '**', '//', '=', '==',
10    '!=', '<', '>', '<=', '>=', '&', '|',
11    '^', '~', '<<', '>>', '(', ')', '[', ']', '{', '}', '.',
12    ',', '!', ':', ';', '"', "'", '\\', '@', '#', '$'
13 ]

```

Listing 3.5: Stopwords Used for Filtering

3.1.1 Machine Learning Model Selection and Architecture

We used a diverse set of Machine Learning models to approach this problem, promising a balanced comparison across multiple aspects. The selected models can be categorized as follows:

1. Linear Models:

- Logistic Regression is effective for binary classification thanks to its simplicity and interpretability. Logistic Regression has been used in vulnerability detection in source code, more specifically for the binary classification of vulnerabilities in source code.

Tang et al. , used Logistic Regression to create an interpretable vulnerability detection model. Their study showed that Logistic Regression could effectively identify vulnerabilities in source code, this served as a basis for assessing the efficacy of other interpretability methods. [10] Similarly, Munonye and Péter utilized Logistic Regression in the detection of vulnerabilities within OAuth 2.0 workflows. Their study emphasized the algorithm's computational efficiency and interpretability, that made it an effective choice for identifying vulnerabilities in sophisticated authentication and authorization workflows. [15]

2. **Tree-Based Models:**

- Random Forest and Gradient Boosting Classifier were employed. These models have been applied for vulnerability detection in source code thanks to their ability in handling non-linear relationships and estimating feature importance.

Rajapaksha et al. demonstrated that Random Forest performs rather well in binary classification tasks, however, Gradient Boosting outperforms it in multi-class classification [20].

Pakshad et al. utilized these models to identify vulnerable functions by analyzing properties in the structure of code, assisting in targeted security testing [17]. Another study focused on the effectiveness of these classifiers in leveraging Explainable AI for understanding vulnerable code [19].

3. **Support Vector Machines (SVM):**

- Support Vector Machines (SVMs) have a certain amount of flexibility allowing them to handle both linear and non-linear data distributions utilizing various kernel functions. Linear kernels are suitable for datasets where classes are linearly separable, effectively finding a hyperplane that maximizes the margin between classes. Non-linear kernels, for more complex datasets, such as the Radial Basis Function (RBF) are used to capture nuanced decision boundaries by mapping

input features into higher-dimensional spaces. This flexibility allows SVMs to model complex relationships within data, making them effective in various applications, including vulnerability detection in source code.

Mouine et al. utilized SVMs with various kernel-based approaches and embedding techniques, such as Bag-of-Words and FastText, to analyze Java source code. Their results demonstrated that SVMs are capable of identifying vulnerabilities effectively, capturing complex patterns in structured datasets, though they were generally outperformed by models such as Random Forest in specific scenarios. [14]

4. **Naive Bayes:**

- **Multinomial Naive Bayes** has been effectively used in text-related tasks due to its reliance on token frequency distributions. Anantawijaya and Mauritsius demonstrated its efficacy in vulnerability detection by analyzing PHP code in the Senayan Library Management System. Using the Bag-of-Words approach for vectorization, their model achieved a remarkable accuracy of 97%, demonstrating the utility of Multinomial Naive Bayes in identifying code vulnerabilities. [2]

5. **Ensemble Methods and AutoML:**

- **XGBoost**, a gradient boosting-based ensemble method, was utilized for its high performance in tabular data tasks.
- **FLAML (AutoML)** was employed for automated model selection and hyperparameter tuning, ensuring optimal configurations.

Chapter 4

Training Process

The training process was designed to systematically prepare, train, and optimize machine learning models for classifying API authentication source code as secure or insecure. It included data splitting, feature representation, model selection, hyperparameter tuning, and finally, evaluation.

4.1 Data Splitting

The dataset was split into training and testing subsets using an 80:20 ratio 4.1, ensuring that 80% of the data was used for training the models and 20% was reserved for evaluating model performance. The split ensured the models were tested on unseen data, reducing the risk of overfitting.

```
1 from sklearn.model_selection import train_test_split
2
3 # Splitting the dataset
4 X_train, X_test, y_train, y_test = train_test_split(
5     vectors, labels, test_size=0.2, random_state=42
6 )
```

Listing 4.1: Data Splitting Process

4.2 Feature Representation

Feature representation involved converting raw Python code into numerical representations using two techniques namely, Count Vectorization and Term Frequency-

Inverse Document Frequency (TF-IDF) 4.2. Count Vectorization counts the frequency of each token in the snippet creating a matrix representation in order to capture term frequencies for a relatively straightforward analysis. These raw frequencies may however not provide enough context for the models. TF-IDF Adjusts aforementioned frequencies by considering their presence across all code snippets, adding an emphasis on unique and relevant terms whilst reducing the weight of common ones. This helps highlight important and unique terms that may have a better impact on the prediction of class labels, such as insecure or secure code. While both methods were utilized in this study, TF-IDF generally provides better feature weighting, which often results in better model performance, particularly when dealing with highly imbalanced data like vulnerabilities in code.

```
1 from sklearn.feature_extraction.text import CountVectorizer
   , TfidfVectorizer
2
3 # Count Vectorization
4 vectorizer = CountVectorizer()
5 count_vectors = vectorizer.fit_transform(X_train)
6
7 # TF-IDF Vectorization
8 tfidf_vectorizer = TfidfVectorizer()
9 tfidf_vectors = tfidf_vectorizer.fit_transform(X_train)
```

Listing 4.2: Vectorization Techniques

4.3 Model Training and Hyperparameter Tuning

Several supervised machine learning models were trained and optimized, including:

- **Logistic Regression:** A linear classifier providing a baseline for performance.
- **Random Forest:** An ensemble model combining multiple decision trees to improve prediction robustness.
- **Gradient Boosting:** A sequential ensemble method that corrects errors from previous iterations.

- **Support Vector Machines (SVM):** A classifier aimed at finding an optimal hyperplane for separating classes.
- **XGBoost:** An advanced boosting algorithm that builds strong models by combining weak learners in a gradient-boosting fashion.
- **Voting Classifier:** An ensemble method that combines predictions from multiple models to improve overall accuracy.
- **FLAML (AutoML):** A framework that automatically selects the best model and tunes hyperparameters to maximize performance.

Hyperparameter tuning was performed using Grid Search and Random Search to identify the optimal configuration for each model. For example in 4.3, the following code illustrates tuning for the Gradient Boosting model:

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.ensemble import GradientBoostingClassifier
3
4 # Defining parameter grid
5 param_grid = {
6     'n_estimators': [50, 100, 150],
7     'learning_rate': [0.01, 0.1, 0.2],
8     'max_depth': [3, 5, 7]
9 }
10
11 # Grid Search
12 grid_search = GridSearchCV(GradientBoostingClassifier(),
13                             param_grid, cv=3, verbose=2)
14 grid_search.fit(X_train, y_train)
15
16 # Best parameters and model
17 best_model = grid_search.best_estimator_

```

Listing 4.3: Hyperparameter Tuning with Grid Search

4.4 Evaluation Metrics

To assess the effectiveness of the models, we used several performance metrics: **accuracy**, **precision**, **recall**, **F1-score**, and the **confusion matrix** 4.4. These metrics provide a comprehensive view of how well the model performs in terms of

both correctly classifying secure and insecure code and minimizing errors. Accuracy measures the overall proportion of correct predictions, while precision focuses on how many of the predicted positive instances were actually positive. Recall, assesses how well the model identifies all true positive instances. The F1-score, balances precision and recall, it is especially useful in cases where the dataset is imbalanced. Finally, the confusion matrix provides a breakdown of true positives, false positives, true negatives, and false negatives, allowing for more detailed error analyses.

```
1 from sklearn.metrics import classification_report,
   accuracy_score, confusion_matrix
2
3 # Making predictions and evaluating
4 y_pred = best_model.predict(X_test)
5 print("Accuracy:", accuracy_score(y_test, y_pred))
6 print("Classification_Report:\n", classification_report(
   y_test, y_pred))
7 print("Confusion_Matrix:\n", confusion_matrix(y_test,
   y_pred))
```

Listing 4.4: Model Evaluation Example

The models were trained on the training dataset using the vectorized features and evaluated on the test dataset. The training process was repeated multiple times to ensure that the results were consistent and not biased by random initialization or other variations. Once the models were trained, predictions were made on the test set, and the performance was measured using the aforementioned evaluation metrics.

We followed a systematic approach for optimal model performance. Data splitting was used for an unbiased evaluation, feature representation techniques such as TF-IDF and Count Vectorization were used to enable certain models to process the text data. We used an exhaustive process spanning a wide set of hyperparameters to maintain a thorough approach. We finally evaluated the performance based on multiple metrics as well as a confusion matrix to be able to view performance through a multitude of perspectives.

Chapter 5

Results

This section showcases and discusses the performance of our machine learning models. Our evaluation metrics include accuracy, precision, recall, F1-Score as well as Confusion Matrices. The goal of all these matrices is to provide an enhanced understanding of the efficacy as well as performance of aforementioned models.

5.1 Logistic Regression

The logistic regression model demonstrated somewhat limited capability in distinguishing between Error and Good classes achieving a test accuracy of 50%, as seen in the balanced but low precision and recall values. The equal distribution of false positives and false indicate that the model does not favor one class over the other but struggles overall in accurate classification.

5.2 Random Forest Classifier

The Random Forest Classifier showed a stronger ability to detect the 'Error' class, shown by its higher recall and F1-Score. The model achieved a test accuracy of 66.7%, However, its performance in identifying the 'Good' class was relatively less robust, with lower recall and F1-Score values. This may be attributed to a class imbalance, in terms of length for our dataset, or simply the model's tendency to favor the 'Error' class.

5.3 Support Vector Machine (SVM)

The SVM with radial basis function (RBF) kernel achieved a test accuracy of 66.7% having a somewhat balanced performance between the 'Error' and 'Good' classes. While comparable to Random Forest in terms of accuracy, it demonstrated better precision for the 'Error' class.

5.4 Gradient Boosting Classifier

The Gradient Boosting Classifier demonstrated robust performance, with a test accuracy of 75% after carefully tuning its hyperparameters. This model was able to effectively capture non-linear dependencies in the data. The precision remained balanced across classes. The disparity in recall highlights areas for improvement, more specifically in detecting "Good" instances of code.

5.5 Extra Trees Classifier

The Extra Trees Classifier emerged as the best-performing model with a test accuracy of 83.3%. It demonstrated strong precision and recall for the "Error" class, with an adequate performance for the "Good" class. Its ability to leverage ensemble techniques and random feature selection contributed significantly to its high performance.

5.6 XGBoost Classifier

XGBoost achieved a test accuracy of 66.7%, showcasing its strong potential when combined with effective hyperparameter tuning. It had a precision of 0.75 for both "Error" and "Good" classes, indicating that 75% of the instances predicted as each class were correct. This suggests the model is equally cautious in predicting both classes, minimizing false positives. Despite its high computational cost, the model performed rather well in precision for the "Good" class.

5.7 FLAML AutoML

FLAML AutoML verified the Extra Trees Classifier as the best-performing model with a test accuracy of 83.3%. Its automated hyperparameter optimization and model selection capabilities really streamlined our evaluation process. The model, after FLAML, configuration had a precision of 1.00 for the "Good" class and a recall of 1.00 for the 'Error' class. This means that it successfully predicted all "Good" and all "Error" instances accurately.

5.8 Summary of Findings

The Extra Trees Classifier and Gradient Boosting emerged as the top performing models, with accuracies of 83.3% and 75%, respectively. Ensemble methods consistently outperformed linear classifiers, highlighting the importance of capturing complex feature interactions 5.1. Future work could explore hybrid models and advanced embeddings to further improve performance.

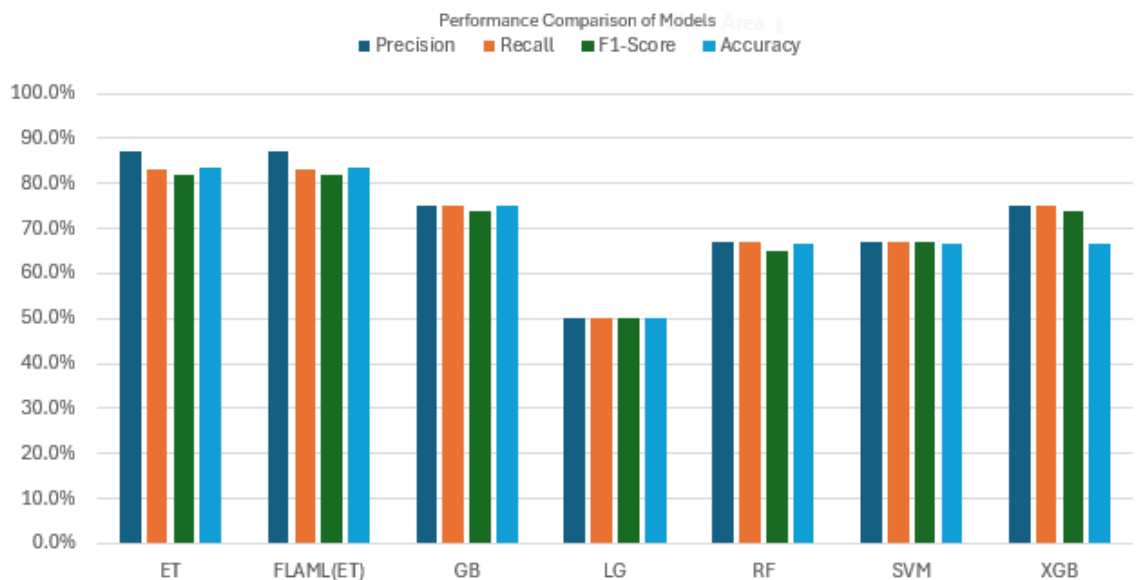


Figure 5.1: Performance Comparison of Models

Chapter 6

Validation through Explainable artificial intelligence (XAI)

This section details the integration of Explainable AI (XAI) techniques into the machine learning pipeline to enhance interpretability and trust in model predictions. The focus is on ensuring that the Extra Trees Classifier, identified as the most effective model, provides transparent decision-making insights for developers and stakeholders.

6.1 Motivation for XAI

Explainable AI bridges the gap between high-performing machine learning models and human understanding. In the context of API vulnerability scanning, XAI helps identify key features contributing to security classifications, making the models more actionable for developers. By integrating XAI, we aim to:

- Identify critical features that influence the classification of secure and insecure code.
- Provide insights into model decision boundaries for improved debugging and optimization.
- Enhance stakeholder trust by demonstrating model accountability.

6.2 SHAP for Feature Importance Analysis for Vulnerability Diagnosing

SHapley Additive exPlanations (SHAP) was employed to quantify feature importance and visualize model interpretability. SHAP values provide additive feature attributions, indicating the contribution of each token to the prediction outcome. It is based on cooperative game theory and assigns each input feature a “contribution value” that shows how much it added to the final prediction 6.1. In simple terms, SHAP helps us understand how each feature influenced a model’s decision. It does this by comparing what the model predicts with and without each feature across all possible combinations. This gives a fair and consistent way to distribute credit (or blame) to the features for the model’s output. This technique is particularly suited for ensemble methods like Extra Trees Classifier.

Implementation Details

```
1 import shap
2 import pickle
3
4 # Load the trained model
5 with open('extra_tree_classifier.pkl', 'rb') as f:
6     model = pickle.load(f)
7
8 # Initialize SHAP TreeExplainer
9 explainer = shap.TreeExplainer(model)
10
11 # Compute SHAP values
12 shap_values = explainer.shap_values(X_test)
13 shap.summary_plot(shap_values, X_test, feature_names=
    feature_names)
```

Listing 6.1: SHAP Integration for Feature Analysis

1. **Preprocessing:** Code snippets were preprocessed to remove comments, filter out stopwords, and tokenize the content. These steps were based on the Dataset preparation activities during Model Training stage. This is important as the dataset were transformed into numerical values. For interpretability we need to convert it back into keywords.

2. **Model Loading:** The Extra Trees Classifier model was loaded using the `pickle` library, and predictions were generated for a subset of the dataset.
3. **SHAP Integration:** The SHAP library was used to calculate feature contributions for individual predictions. The `TreeExplainer` module was utilized to interpret the ensemble model.

Feature Importance Results

Figure 6.1 presents the SHAP summary plot, highlighting the most influential features. Tokens such as `'authorization'`, `'admin'`, and `'token'` were identified as key indicators for distinguishing between secure and insecure API code. These findings align with domain knowledge, emphasizing the role of authentication mechanisms in API vulnerabilities.

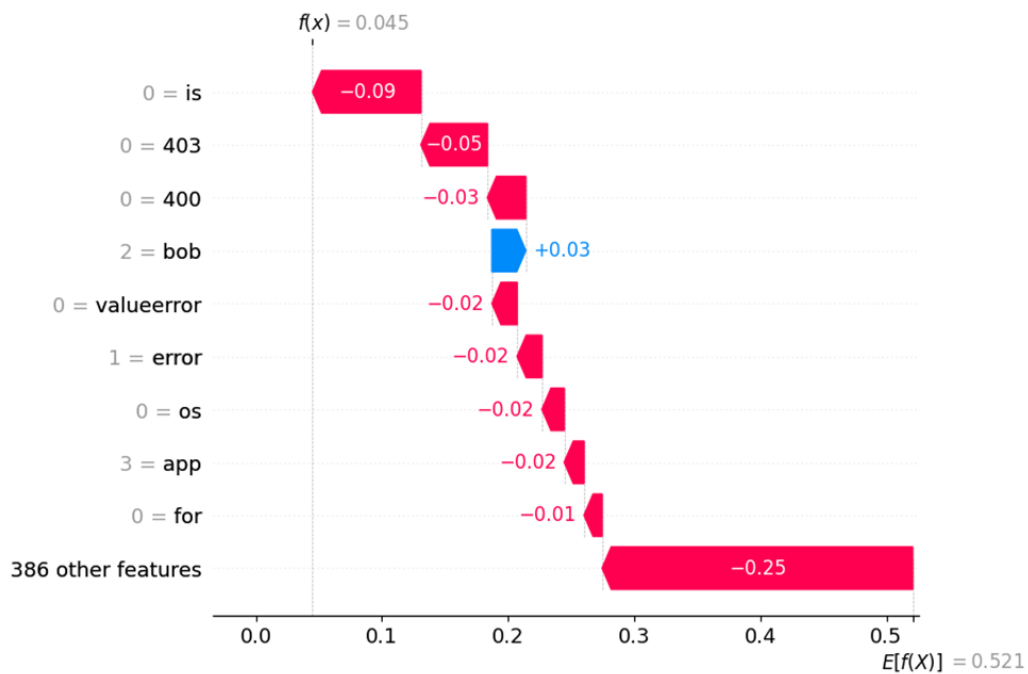


Figure 6.1: SHAP Summary Plot for Feature Importance

6.3 Impact of XAI on Model Optimization for Vulnerability Diagnosing

The SHAP waterfall diagram provides specific insights for the individual contributions of source code features for a vulnerability prediction. In this example, the predicted output is represented as $E[f(x)] = 0.521$, while the actual contribution of features yields $f(x) = 0.045$. This difference underscores the importance of source code feature-level explainability in trying to interpret model behavior.

From the diagram:

- Missing features such as `'is'` (-0.09), `'403'` (-0.05), and `'400'` (-0.03) had significant negative impacts on the prediction.
- The feature `'bob'` contributed positively ($+0.03$) to the prediction.
- Other features, including `'valueerror'` (-0.02), `'error'` (-0.02), and `'os'` (-0.02), showed minor negative impacts.
- A large number of features (386 in total) collectively contributed to a decrease of -0.25 in the prediction value, indicating the cumulative effect of low-impact features.

6.4 Discussion and Conclusion

The integration of SHAP-based explainable AI (XAI) techniques into the model evaluation process has proven invaluable. The transparent visualization of feature contributions, as seen in the waterfall diagram, facilitates better decision-making in model refinement. For instance, excluding tokens with low SHAP contributions from the input space reduces noise and enhances generalizability. Furthermore they provide insights into how these models are making their predictions and which aspects of the source code are impacting these predictions.

Furthermore, emphasizing important tokens (e.g., those related to security concepts like `'authorization'` and `'token'`) ensures that the model prioritizes key aspects of the input data. This approach not only improves interpretability but also builds trust in AI systems by offering a clear understanding of the factors influencing predictions. Consequently, these advancements support the development of robust API vulnerability detection frameworks and other critical AI applications.

Chapter 7

Discussion

This study aimed to integrate **Explainable AI (XAI)** techniques into machine learning models for the detection of vulnerabilities in API authentication code. The results of our experiments, involving various machine learning models like Random Forest, Extra Trees, Support Vector Machines (SVM), and XGBoost, highlight the importance of combining both **model performance** and **interpretability** in real-world applications like security.

7.1 Model Performance Evaluation

The models tested in this study demonstrated varying degrees of success, with the **Extra Trees Classifier** performing the best, achieving an accuracy of 83.3%. This model leveraged the power of ensemble learning and benefited from fine-tuning, particularly when compared to simpler models like Logistic Regression and Naive Bayes, which showed relatively lower performance (e.g., Logistic Regression achieved only 50% accuracy).

While tree-based models, including **Random Forest** and **Gradient Boosting**, were more robust in handling the feature interactions present in the dataset, other models like **SVM** and **XGBoost** also showed promising results. These models exhibited high precision, particularly in the “Error” class, making them quite useful for such specific types of vulnerability detection.

A key takeaway from this experiment is that ensemble methods tend to outperform single classifiers thanks to their ability to capture complex interactions between code features, which is particularly valuable in security-related tasks where vulnerabilities are often nuanced and context-dependent.

7.2 Impact of Explainable AI

Incorporating **SHAP** for **feature importance** analysis provided a window into how these models make predictions. By leveraging **XAI techniques**, we were able to identify critical features, source code terms, that significantly influenced model decisions. This is essential, as understanding which features are most impactful can help security teams refine API code, highlighting areas that are most prone to vulnerabilities. This can then further be utilized in coding policies and vulnerability prevention.

Despite the success of XAI, we acknowledge the computational complexity introduced by these techniques. While **SHAP** offers precise and detailed insights, its computational cost may increase with the size and complexity of the dataset. This trade off must be considered when deploying XAI methods in real-time applications, especially in production environments where speed is critical and resources are minimal.

Chapter 8

Limitations and Future Work

While the study demonstrates promising results, several limitations should be addressed in future work:

Dataset Limitation: The dataset used in this study contained only 60 labeled code snippets. Although it was sufficient for initial exploration, a larger and more diverse dataset would be necessary to validate the results and improve model robustness.

Model Variability: While ensemble models like **Extra Trees** performed well, other models like **XGBoost** and **SVM** showed inconsistent performance. Further tuning and experimentation with different hyperparameter configurations might help enhance their stability and predictive power.

Scalability of XAI Techniques: As the dataset grows, the **computational burden** of XAI methods like **SHAP** may increase. Exploring other model agnostic interpretability techniques, such as **LIME**, or improving the scalability of **SHAP** would be valuable.

Hybrid Models: Future work could explore combining the strengths of both **unsupervised** and **supervised** learning to create hybrid models that can effectively detect vulnerabilities while maintaining interpretability.

Chapter 9

Conclusion

This research demonstrates the potential of **Explainable AI (XAI)** techniques in enhancing the interpretability and trustworthiness of machine learning models used for **API vulnerability detection**. By integrating **SHAP**, we were able to uncover valuable insights into how the models identify critical features that influence the classification of secure and insecure API authentication code.

Our findings indicate that **ensemble models** like **Extra Trees Classifier** and **Random Forest** are particularly well-suited for vulnerability detection, as they can capture complex feature interactions more effectively than simpler models. However, while **XAI** techniques such as **SHAP** provide transparency, they also introduce **computational challenges** that need to be addressed for large-scale deployment.

This study highlights the importance of interpretability in security-related AI applications, providing developers with actionable insights into the features driving vulnerability predictions. By combining high-performing models with transparent, explainable techniques, we can not only improve model accuracy but also ensure that security decisions are well-understood, trusted, and actionable.

In summary, the integration of **XAI** methods enhances **AI-driven security solutions** by fostering trust, providing transparency, and ultimately guiding developers in building more secure systems. As AI models become more prevalent in critical applications like security, the need for **explainable** and **interpretable** solutions will continue to grow, indicating significant potential for future research and development.

References

- [1] OWASP® Foundation Employee Directory, Headcount & Staff | LeadIQ. URL <https://leadiq.com/c/owasp-foundation/5a1d96702300005200850ff0/employee-directory?page=2>.
- [2] I Komang Sughosa Anantawijaya and Tuga Mauritsius. Detecting vulnerabilities in senayan library management system using machine learning. Journal of Logistics, Informatics and Service Science, 11(6):266–278, 2024. doi: 10.33168/JLISS.2024.0615. URL <https://doi.org/10.33168/JLISS.2024.0615>.
- [3] Sanjay Bahl and O. P. Wali. Perceived significance of information security governance to predict the information security service quality in software service industry: An empirical analysis. Information Management & Computer Security, 22(1):2–23, March 2014. ISSN 0968-5227. doi: 10.1108/IMCS-01-2013-0002. URL <https://www.emerald.com/insight/content/doi/10.1108/imcs-01-2013-0002/full/html>. Publisher: Emerald Group Publishing Limited.
- [4] H. K. Brar and P. J. Singh. Comparing detection ratio of three static analysis tools. International Journal of Computer Applications, 124(13):1–5, 2015. doi: 10.5120/ijca2015905749.
- [5] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Zhiqiang Lin, and Yinqian Zhang. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In RAID ’23 Proceedings, pages 23–23, 2023. doi: 10.1145/3607199.3607242.
- [6] F5 Networks, Inc. Continuous API Sprawl, November 2021. <https://www.f5.com/pdf/reports/>

f5-office-of-the-cto-report-continuous-api-sprawl.pdf.

- [7] OWASP Foundation. Owasp api security top ten, 2023. URL <https://owasp.org/www-project-api-security-top-10/>. Accessed: 2024-12-24.
- [8] Gartner. Predicts 2022: Apis demand improved security and management, December 2021. URL <https://www.gartner.com/en/documents/4009103>. Accessed: 2024-11-10.
- [9] Danielle Gonzalez, Paola Peralta Perez, and Mehdi Mirakhorli. Barriers to shift-left security: The unique pain points of writing automated tests involving security controls. In Proceedings of the 15th ACM / IEEE International Symposium on empirical Software Engineering and Measurement (ESEM), ESEM '21, page 11, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450386654. doi: 10.1145/3475716.3475786. URL <https://doi.org/10.1145/3475716.3475786>.
- [10] Ashish Kumar, Rachit Agarwal, Srinivas R., and et al. Interpretation of learning-based automatic source code vulnerability detection model using lime. IEEE Transactions on Software Engineering, 47(12):2412–2426, 2021. doi: 10.1109/TSE.2021.3091360. URL <https://ieeexplore.ieee.org/document/9441102>.
- [11] Zhen Li, Deqing Zou, and Shouhuai Xu. Vuldeepecker: An automated vulnerability detection system based on code similarity analysis. In Proceedings of the 2018 ACM Conference, pages 200–210, 2018.
- [12] Ruitong Liu, Yanbin Wang, Haitao Xu, Bin Liu, Jianguo Sun, Zhenhao Guo, and Wenrui Ma. Source code vulnerability detection: Combining code language models and code property graphs. arXiv preprint arXiv:2404.14719, 2023. URL <https://arxiv.org/abs/2404.14719>.
- [13] Tina Marjanov, Ivan Pashchenko, and Fabio Massacci. Machine learning for source code vulnerability detection: What works and what isn't there yet. IEEE Security & Privacy, 20(5):60–69, 2022. doi: 10.1109/MSEC.2022.3176058.
- [14] Esma Mouine, Yan Liu, Lu Xiao, Rick Kazman, and Xiao Wang. Explaining the contributing factors for vulnerability detection in machine learning.

arXiv preprint arXiv:2406.03577, 2024. URL <https://arxiv.org/abs/2406.03577>.

- [15] Kindson Munonye and Martinek Péter. Machine learning approach to vulnerability detection in oauth 2.0 authentication and authorization flow. International Journal of Information Security, 21(2):223–237, 2022. doi: 10.1007/s10207-021-00551-w. URL <https://doi.org/10.1007/s10207-021-00551-w>.
- [16] Henry R. Neave. Deming’s 14 points for management: Framework for success. Journal of the Royal Statistical Society. Series D (The Statistician), 36(5):561–570, 1987. ISSN 00390526, 14679884. URL <http://www.jstor.org/stable/2348667>.
- [17] Puya Pakshad, Alireza Shameli-Sendi, and Behzad Khalaji Emamzadeh Abbasi. A security vulnerability predictor based on source code metrics. International Journal of Information Security, 21:315–327, 2023. doi: 10.1007/s11416-023-00469-y. URL <https://link.springer.com/article/10.1007/s11416-023-00469-y>.
- [18] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Ai-powered vulnerability detection for secure source code development. In Innovative Security Solutions for Information Technology and Communications (SecITC 2022), volume 13809 of Lecture Notes in Computer Science, pages 275–288. Springer, 2023. doi: 10.1007/978-3-031-32636-3_16. URL https://link.springer.com/chapter/10.1007/978-3-031-32636-3_16.
- [19] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Enhancing security assurance in software development: Ai-based vulnerable code detection with static analysis. In Proceedings of the Advances in Software Engineering Conference, pages 310–320, 2023. doi: 10.1007/978-3-031-54129-2_20. URL https://link.springer.com/chapter/10.1007/978-3-031-54129-2_20.
- [20] Sampath Rajapaksha, Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. Ai-powered vulnerability detection for secure source code development. In Proceedings of the International Conference on Software Engineering, volume 32, pages 251–260, 2023. doi: 10.

1007/978-3-031-32636-3_16. URL https://link.springer.com/chapter/10.1007/978-3-031-32636-3_16.

- [21] Rebecca L. Russell, Louis Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. Automated vulnerability detection in source code using deep representation learning. In 17th IEEE International Conference on Machine Learning and Applications (IEEE ICMLA 2018), pages 1–8, Orlando, Florida, USA, 2018. URL <https://arxiv.org/abs/1807.04320v2>.
- [22] Md. Shazzad Hossain Shaon, Alfredo Cuzzocrea, Mst Shapna Akter, and Shahriyar Zaman Ridoy. Enstack: An ensemble stacking framework of large language models for enhanced vulnerability detection in source code. arXiv preprint arXiv:2411.16561, 2024. URL <https://arxiv.org/abs/2411.16561>.
- [23] Amit Sheps. Why Shift Right Testing & Security? Shift Right vs. Shift Left, 2024. URL <https://www.aquasec.com/cloud-native-academy/devsecops/shift-right-testing-and-security/>.
- [24] Alexey Vishnyakov, Daniil Kuts, Vlada Logunova, Darya Parygina, Eli Kobrin, Georgy Savidov, and Andrey Fedotov. Sydr-fuzz: Continuous hybrid fuzzing and dynamic analysis for security development lifecycle. In 2022 Ivannikov Ispras Open Conference (ISPRAS), pages 111–123, 2022. doi: 10.1109/ISPRAS57371.2022.10076861.
- [25] Mohammad Wazid, Ashok Kumar Das, Vinay Chamola, and Youngho Park. Uniting cyber security and machine learning: Advantages, challenges and future research. ICT Express, 8(3):313–321, September 2022. ISSN 2405-9595. doi: 10.1016/j.icte.2022.04.007. URL <https://www.sciencedirect.com/science/article/pii/S2405959522000637>.
- [26] David A. Wheeler. Flawfinder. <https://dwheeler.com/flawfinder/>, 2024. Accessed: 2024-12-11.
- [27] Fiorella Zampetti, Simone Scalabrino, Rocco Oliveto, Gerardo Canfora, and Massimiliano Di Penta. How open source projects use static code analysis tools in continuous integration pipelines. In 2017 IEEE/ACM 14th

International Conference on Mining Software Repositories (MSR), pages
334–344, 2017. doi: 10.1109/MSR.2017.2.